

ipythonintro

January 23, 2026

1 Introduction to python, ipython, & numpy

This document is meant to give a brief introduction to python, ipython, and numpy. There are many useful tutorials online, and students are encouraged to seek these out as needed.

First, a few introductions: - Python is a programming language. We will be doing everything in python in this class. - iPython stands for interactive python. It is a way of writing python code that allows you to run things interactively, rather than having a file that you run. It's similar to the interactive command line prompt in Matlab. - Jupyter notebook used to be called the iPython notebook. It is just a re-branding. It allows you to run code from a browser window. - Numpy and scipy are python packages for scientific computing. If it's a common math operation, these packages probably have a function that does it.

This is a special command that tells jupyter notebook to render all plots and images in the notebook, rather than saving it to file. You always want to add this at the beginning of your notebook.

```
[3]: %matplotlib inline
```

To run a cell, you can simply press SHIFT + ENTER.

Note that there are two different kinds of cells: code cells and markdown cells. Markdown cells are primarily for showing text. You can easily create bulleted lists and section headers in markdown cells. Look up tutorials on markdown for more info.

1.0.1 Python basics

Create variable, list, and dictionary.

```
[4]: a = 3
      b = a + 2
      b
```

```
[4]: 5
```

```
[5]: a = [2,3,4]
      for i in a: # iterate over each element of a
          x = i*2
          print(x)
```

4
6
8

Note that indexing starts from 0.

```
[6]: a[0] = 6  
a
```

```
[6]: [6, 3, 4]
```

This is a very common idiom for iterating over a range of numbers.

```
[7]: for i in range(5):  
      print(i)
```

0
1
2
3
4

Note that tuples are not the same as lists, though they have some common functionality. Tuples are usually used for packaging a small number of things together. Lists are used for longer sequences of things, and when you need to be able to delete or add new elements.

```
[8]: a = (1,2,3) # tuple  
      b = [1,2,3] # list  
      a[0], b[1]
```

```
[8]: (1, 2)
```

You can append things to a list using the append function.

```
[9]: b.append(4)  
      b.append(5)  
      b
```

```
[9]: [1, 2, 3, 4, 5]
```

A dictionary allows you to look things up efficiently.

```
[10]: a = {}  
      a['test'] = 3 # 'test' is the key, 3 is the value  
      a['abc'] = 2  
      a
```

```
[10]: {'test': 3, 'abc': 2}
```

To test if a key exists in the dictionary:

```
[11]: if 'abcd' in a:
        print('abcd is in the dictionary')
    else:
        print('abcd is not in the dictionary')
```

abcd is not in the dictionary

```
[12]: 'abc' in a
```

```
[12]: True
```

```
[13]: a['test'] = 1
      a
```

```
[13]: {'test': 1, 'abc': 2}
```

```
[14]: a['test']
```

```
[14]: 1
```

To define a function, you can use the following basic structure.

```
[15]: def myFunction(x, y): # x and y are arguments of the function
        '''
        Comments are usually put here at the beginning of the function.
        '''
        result = x + y
        return result
```

To call the function, you can do the following. Remember to run the cell containing the function definition before you call it!

```
[16]: a = myFunction(3,5)
      a
```

```
[16]: 8
```

Functions can call other functions. You can also return more than one result.

```
[17]: def otherFunction(x,y,z=5): # the z=5 is a way of setting default values if not
        ↪provided
        w = myFunction(y,z)
        result = w - x
        return (result, w)
```

```
[18]: a, b = otherFunction(2,3,4)
      a, b
```

```
[18]: (5, 7)
```

```
[19]: a, b = otherFunction(2,3) # use default value for z argument
      a, b
```

```
[19]: (6, 8)
```

1.0.2 Numpy & Scipy

To import a package, you can do the following. Note that normally you would import all the packages and functions you need at the beginning of your notebook in a single cell.

```
[20]: import numpy as np
      import os.path
```

```
[21]: a = '/path/to/file.txt'
      os.path.basename(a)
```

```
[21]: 'file.txt'
```

You can also import specific functions if you want to keep your code more readable.

```
[22]: from os.path import basename
```

```
[23]: basename(a)
```

```
[23]: 'file.txt'
```

If you want to do something that is commonly done (e.g. matrix multiplication, FFT, extracting the basename from a file path), you can google it (e.g. “numpy FFT”), import the package or function, and then call the function. Online documentation is your friend.

numpy has a vast set of useful functions for working with numerical data. Note that a numpy array – which is used to represent vectors and matrices – is different from a python list. They have some methods in common, but if you are working with numerical data, you usually want to use a numpy array.

```
[24]: a = [1,2,3,4] # python list
      b = np.array([1,2,3,4]) # numpy array
      b
      a
```

```
[24]: [1, 2, 3, 4]
```

You can reshape 1-D arrays of numbers into matrices like so:

```
[25]: c = np.arange(20).reshape((4,5))
      c
```

```
[25]: array([[ 0,  1,  2,  3,  4],
            [ 5,  6,  7,  8,  9],
            [10, 11, 12, 13, 14],
            [15, 16, 17, 18, 19]])
```

```
[26]: c.shape
```

```
[26]: (4, 5)
```

You can access elements from a numpy array using notation very similar to matlab. Note that indexing starts from 0.

```
[27]: c[1,2] # single element
```

```
[27]: 7
```

```
[28]: c[:,1] # all rows from column index 1
```

```
[28]: array([ 1,  6, 11, 16])
```

```
[29]: c[:,1:3] # all rows from column indices 1 and 2
```

```
[29]: array([[ 1,  2],
            [ 6,  7],
            [11, 12],
            [16, 17]])
```

```
[30]: c[1:3,2:5]
```

```
[30]: array([[ 7,  8,  9],
            [12, 13, 14]])
```

Numpy provides helpful ways to access elements at the end

```
[66]: c
```

```
[66]: array([[1., 0., 0., 0.],
            [0., 1., 0., 0.],
            [0., 0., 1., 0.],
            [0., 0., 0., 1.]])
```

It also provides convenient ways to consider every N^{th} row or column:

```
[98]: c[:,0::1] # all rows, every other column starting at column index 1
```

```
[98]: array([[1., 0., 0., 0.],
            [0., 1., 0., 0.],
            [0., 0., 1., 0.]])
```

```
[0., 0., 0., 1.]])
```

Note that `*` does elementwise multiplication.

```
[33]: a = np.arange(4).reshape((2,2))  
      b = np.arange(4).reshape((2,2))
```

```
[34]: a
```

```
[34]: array([[0, 1],  
            [2, 3]])
```

```
[35]: a * b
```

```
[35]: array([[0, 1],  
            [4, 9]])
```

To do matrix multiplication, we google “numpy matrix multiply”, read the documentation, and call the function accordingly. The `@` operator is a shortcut for matrix multiplication.

```
[36]: c = np.matmul(a,b)  
      c
```

```
[36]: array([[ 2,  3],  
            [ 6, 11]])
```

```
[37]: a@b
```

```
[37]: array([[ 2,  3],  
            [ 6, 11]])
```

1.0.3 Broadcasting

A very important concept when working with numpy is broadcasting. Numpy is smart about inferring your intent when working with scalars and arrays, and it will modify the variables accordingly in the background. This allows us to write code that is much more clean and elegant.

```
[38]: a = 1  
      b = np.arange(10)  
      b
```

```
[38]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
[39]: b+1 # broadcasts the scalar 1 to be the same size as b
```

```
[39]: array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

```
[40]: c = np.eye(4)
      c
```

```
[40]: array([[1., 0., 0., 0.],
            [0., 1., 0., 0.],
            [0., 0., 1., 0.],
            [0., 0., 0., 1.]])
```

```
[41]: c + 2 # broadcasts the scalar 2 to be the same size as c
```

```
[41]: array([[3., 2., 2., 2.],
            [2., 3., 2., 2.],
            [2., 2., 3., 2.],
            [2., 2., 2., 3.]])
```

We can also broadcast vectors to matrices.

```
[42]: d = np.arange(4).reshape((-1,1))
      d
```

```
[42]: array([[0],
            [1],
            [2],
            [3]])
```

Note how numpy broadcasts `d` to match the size of `c` when we add and multiply.

```
[78]: c + d # broadcast d to match the size of c
```

```
[78]: array([[1., 0., 0., 0.],
            [1., 2., 1., 1.],
            [2., 2., 3., 2.],
            [3., 3., 3., 4.]])
```

```
[44]: c * d
```

```
[44]: array([[0., 0., 0., 0.],
            [0., 1., 0., 0.],
            [0., 0., 2., 0.],
            [0., 0., 0., 3.]])
```

One note of caution: a numpy array of size `(k,)` is not the same as a numpy array of size `(k,1)`. Be careful about this, especially when doing broadcasting.

```
[45]: c[:,0].shape
```

```
[45]: (4,)
```

```
[79]: d.shape
```

```
[79]: (4, 1)
```

1.0.4 Vectorization

Numpy is optimized for vector and matrix operations. So, whenever possible, you should try to remove for loops.

Exercise. Let's say you are trying to generate an $N \times N$ matrix whose element values are the sum of the row and column indices. Generally, you want to pre-allocate numpy arrays and set the values accordingly, rather than dynamically increasing the size of the numpy array.

Bad.

```
[80]: def getMatrix_bad(N):
      A = np.zeros((N,N))
      for i in range(N):
          for j in range(N): # nested for loops are usually bad
              A[i,j] = i+j
      return A
```

```
[81]: A = getMatrix_bad(4) # verify
      A
```

```
[81]: array([[0., 1., 2., 3.],
            [1., 2., 3., 4.],
            [2., 3., 4., 5.],
            [3., 4., 5., 6.]])
```

Better.

```
[89]: 0:4
```

```
Cell In[89], line 1
```

```
0:4
```

```
~
```

```
SyntaxError: illegal target for annotation
```

```
[91]: def getMatrix_better(N):
      A = np.zeros((N,N))
      for i in range(N):
          A[i,:] = i + np.arange(N)
      return A
```

```
[92]: A = getMatrix_better(4) # verify
      A
```



```
[92]: array([[0., 1., 2., 3.],
            [1., 2., 3., 4.],
            [2., 3., 4., 5.],
            [3., 4., 5., 6.]])
```

Can you figure out a way to do this in one line using broadcasting?

```
[93]: def getMatrix_best(N):
      return np.arange(N).transpose + np.arange(N)
```

```
[94]: A = getMatrix_better(4) # verify
      A
```

```
[94]: array([[0., 1., 2., 3.],
            [1., 2., 3., 4.],
            [2., 3., 4., 5.],
            [3., 4., 5., 6.]])
```

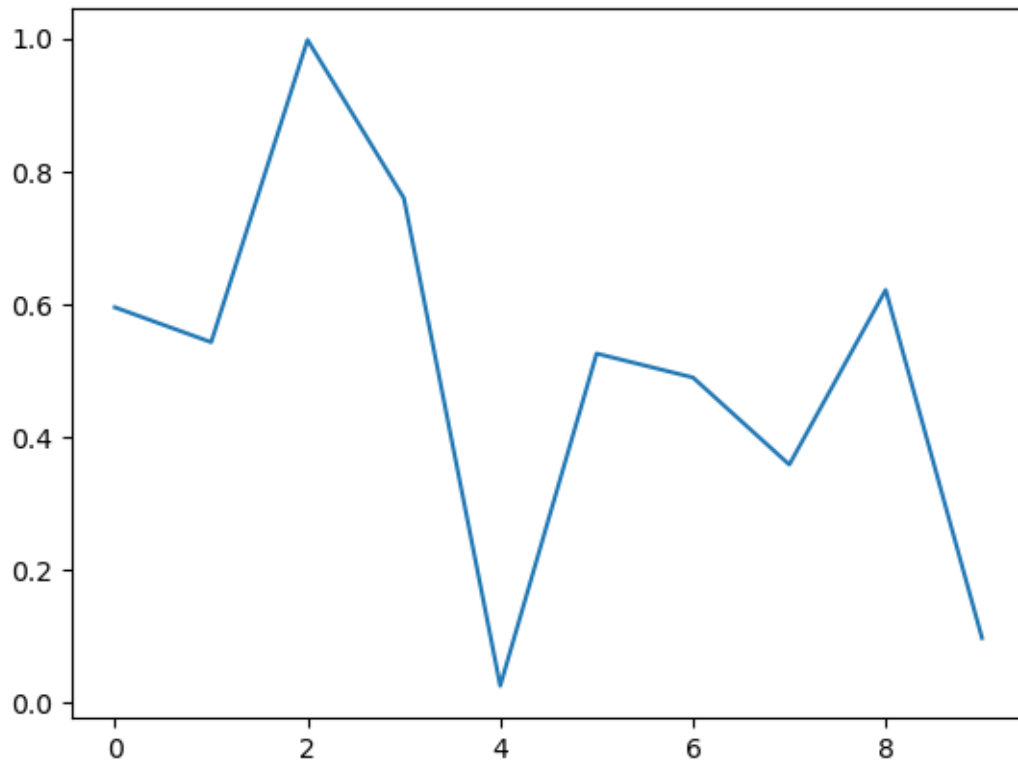
1.0.5 Visualization

A very useful package for plotting is matplotlib.

```
[95]: import matplotlib.pyplot as plt
```

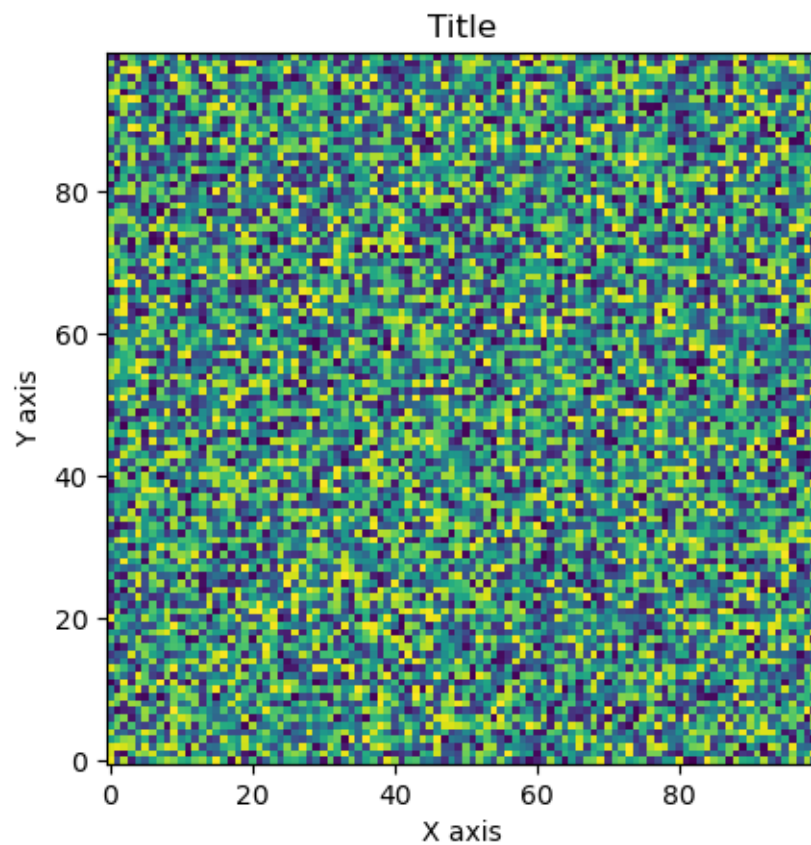
```
[54]: a = np.random.random(10)
```

```
[55]: plt.plot(a)
      plt.show()
```



```
[56]: img = np.random.random((100,100))
```

```
[57]: plt.imshow(img, origin='lower')  
plt.xlabel('X axis')  
plt.ylabel('Y axis')  
plt.title('Title')  
plt.show()
```



To figure out how to plot data in a certain way, it's easiest to google around and find code examples to use as templates.